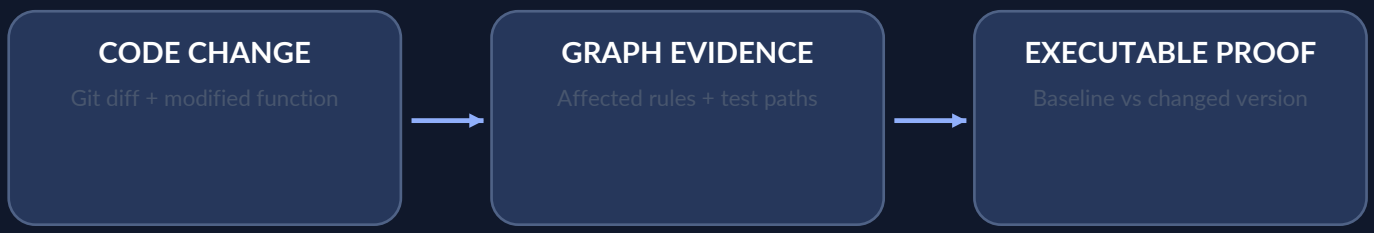
GRAPH HACKS / 2026

TWO-PERSON TEAM / PROJECT CONCEPT

FAULTLINE ZERO

Graph-Native Regression Investigation Agent

Before a code change ships, discover what it may break - then try to reproduce the break with executable evidence.



PRIMARY TRACK

01 / Agents on Connected Data

DOCUMENT STATUS

Proposed / Not Yet Built

Team briefing | Problem statement, system design, flowcharts, MVP, evaluation and submission rules

A specific problem worth solving

Automated test suites can pass even when an important cross-service business rule has been broken.

PROBLEM STATEMENT

Software changes may introduce regressions that are invisible to existing unit tests. Developers lack a practical way to connect the modified code to affected business invariants, select relevant failure scenarios, and obtain reproducible evidence before deployment.

WHY EXISTING WORKFLOWS MISS IT

Fragmented context

Tests know functions; business rules span multiple operations and services.

Speculative signals

A risk score or LLM warning is not the same as an executable regression.

Costly broad testing

Running every scenario after every change can waste time and still miss missing tests.

OUR APPROACH

Represent code components, operations, invariants and scenario tests as connected entities in **FalkorDB**. An investigation agent follows affected paths, selects relevant tests, executes a baseline-versus-change comparison, and reports verified findings with precise graph provenance.

TWO OBJECTIVES

- **Discover:** Trace modified code to meaningful business invariants using graph queries, not keyword matching alone.
- **Verify:** Run controlled test scenarios against baseline and changed implementations; distinguish reproduced regression from suspicion or insufficient evidence.

POSITIONING / IMPORTANT LIMITATION

This is not a claim to invent regression testing or AI code review. The proposed contribution is their **graph-connected, evidence-first integration**. A prototype demonstrates only the scenarios it can execute; passing those does not prove the entire application is safe.

A regression normal tests may miss

Example uses a synthetic payment service and fake gateway - no real transactions, secrets or payment credentials.

BUSINESS INVARIANT

For a given order ID, at most one successful payment capture may be recorded.

BASELINE VERSION

01 Create order ORD-104

02 Gateway captures payment

03 Response to client is lost

04 Client retries with same key

05 Idempotency check prevents a new capture

1 CAPTURE / PASS

CHANGED VERSION

01 Create order ORD-104

02 Gateway captures payment

03 Response to client is lost

04 Client retries after code change

05 Removed/broken guard allows second capture

2 CAPTURES / FAIL

REPRODUCTION TEST - AN INDEPENDENT ORACLE

GIVEN a single order and fake gateway with controlled timeout
WHEN the same payment request is retried after a lost response
THEN **successful_capture_count(order_id) == 1**

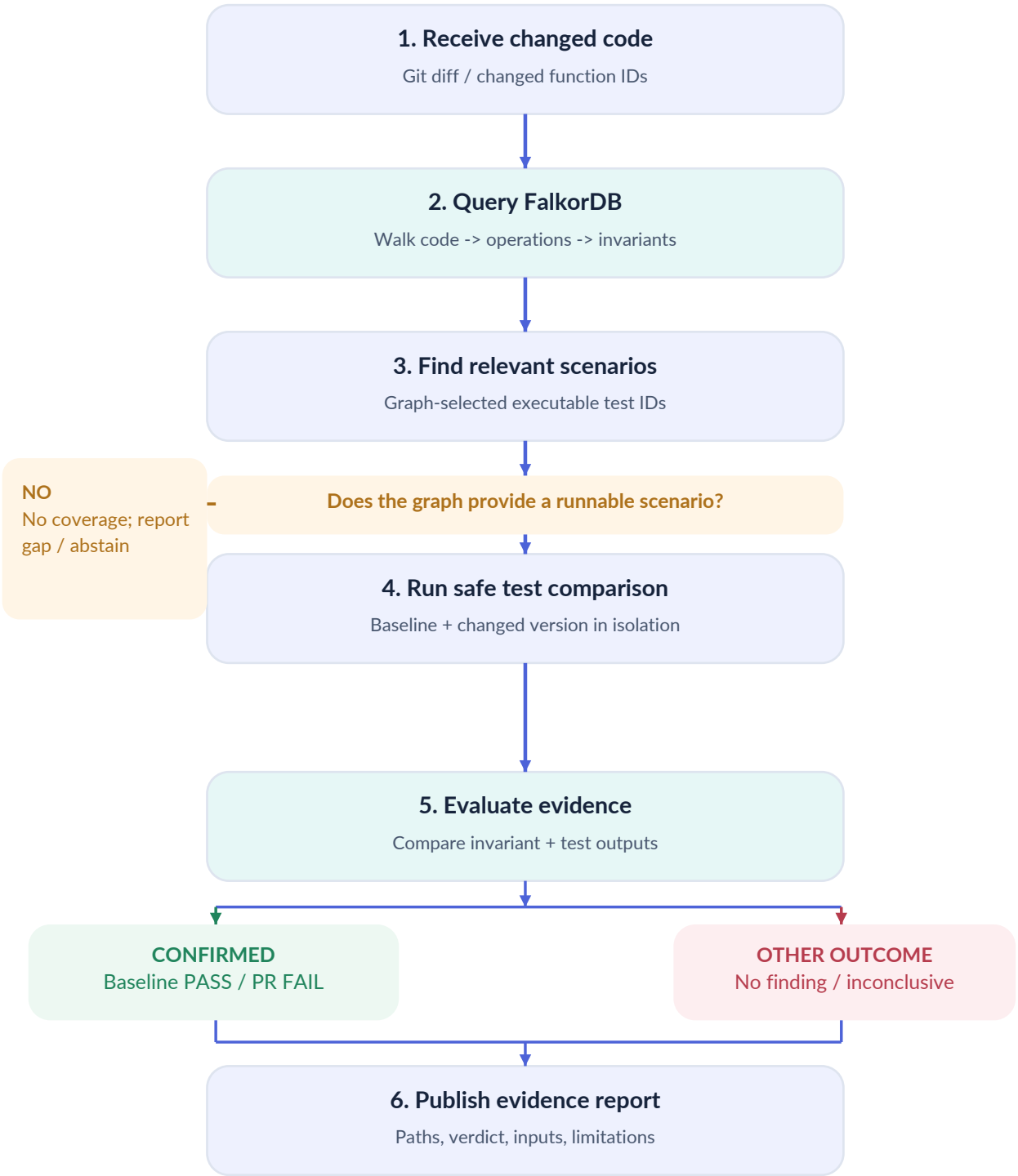
WHAT THE AGENT MUST SHOW

- Graph evidence: **retry_handler -> PaymentService -> capture -> one_capture_per_order -> timeout_retry_test**.
- Execution proof: baseline PASS, changed version FAIL, reproducible input and output; if either test cannot run, mark the result **inconclusive**.

All displayed results on this page are target demo behaviour, not current measurements.

End-to-end investigation flow

The system must make a real graph-guided tool decision before a test can be selected and executed.

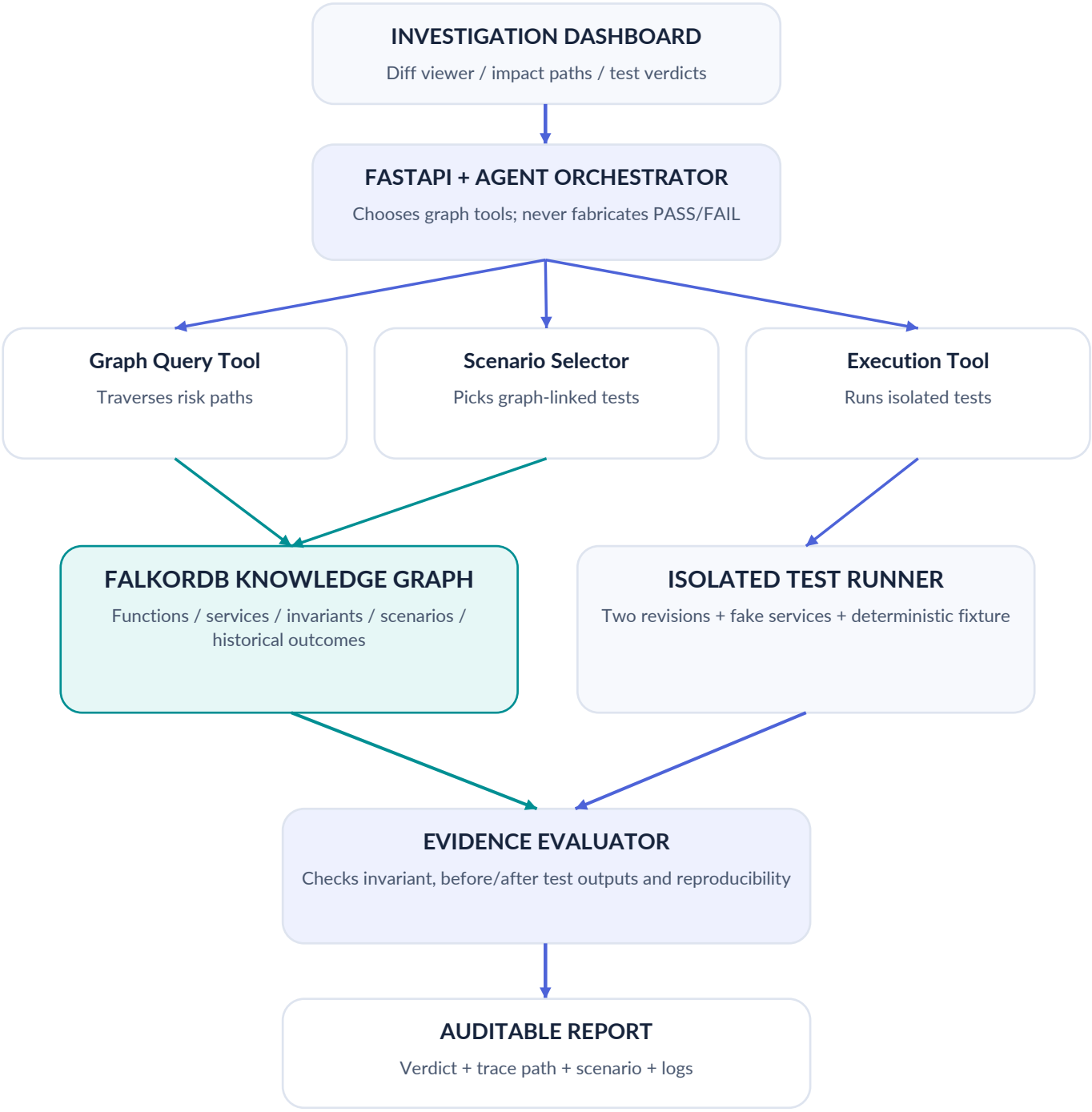


DECISION POLICY

A verified regression requires an independent invariant and a reproduced baseline-pass / changed-fail result. Missing graph coverage, failed setup or nondeterministic output must not be labelled as safe.

Design: graph reasoning + proof

An AI agent proposes investigation steps; deterministic graph queries and test results carry the factual evidence.

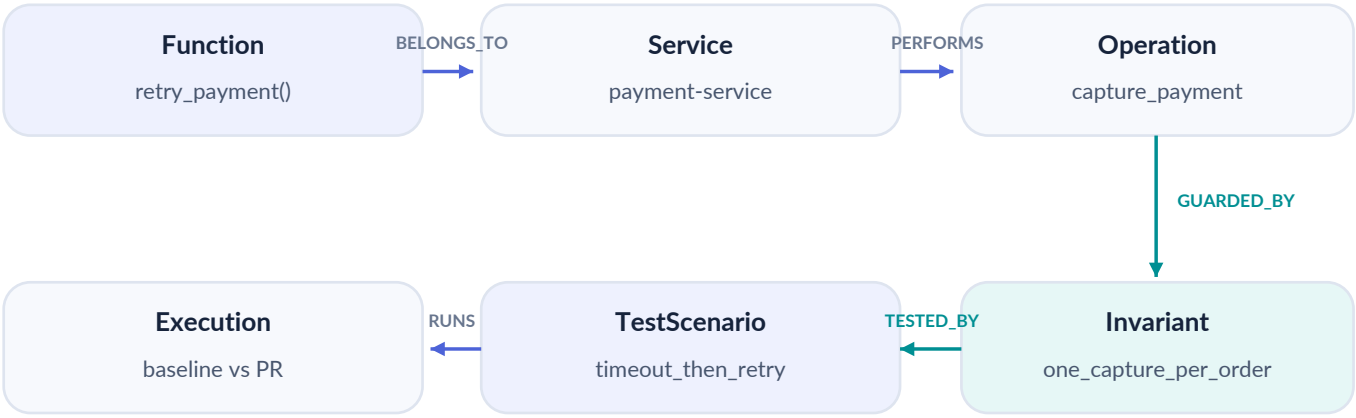


DESIGN PRINCIPLES

- **Graph is load-bearing:** scenario discovery and evidence trails require reachable graph paths.
- **Agent is bounded:** no production deployment, no untrusted arbitrary code execution, no unsupported conclusions.

FalkorDB graph schema & tools

Each node and relationship gives the agent a navigable, queryable reason for choosing a targeted test.



MODEL EXTENSIONS

A **PullRequest** CHANGES Function; a **TestRun** PRODUCES Outcome; an **Investigation** REFERENCES graph paths and stores provenance. The minimum MVP requires version identifiers on test executions.

EXAMPLE CYPHER - ILLUSTRATIVE QUERY

```
MATCH (f:Function {name: $changedFunction})-[:BELONGS_TO]->(s:Service)

MATCH (s)-[:PERFORMS]->(op:Operation)-[:GUARDED_BY]->(inv:Invariant)

MATCH (inv)-[:TESTED_BY]->(test:TestScenario)

RETURN DISTINCT s.name, op.name, inv.id, test.id
```

The query selects runnable scenarios for a changed function by traversing domain relationships. For larger projects, call-graph edges and multi-hop paths extend this model.

AGENT TOOL CONTRACT

read_code_changes
Identify changed function IDs from a diff

find_affected_invariants
Return graph paths, rule IDs, test IDs

run_baseline_vs_changed
Execute allowlisted, isolated scenarios

build_evidence_report
Return classification and reproducible artifacts

Investigation dashboard - UI concept

A single evidence-focused screen: what changed, what the graph found, and what the controlled test proved.

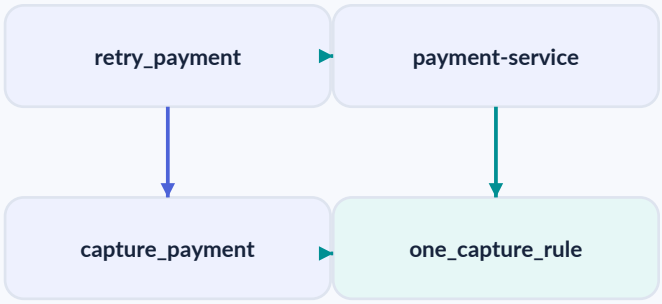
FAULTLINE ZERO — Investigation / PR #24

CODE CHANGE
retry_payment()

GRAPH PATHS
1 invariant / 1 test

VERDICT
REGRESSION

GRAPH EVIDENCE



Trace: Function -> Service -> Operation -> Rule

TEST COMPARISON

BASELINE
PASS / 1 capture

CHANGED
FAIL / 2 captures

Controlled fake gateway test; isolated execution log linked.

RECOMMENDED NEXT ACTION

Hold this example deployment for review. Investigate idempotency handling and attach the reproducible test output to the pull request.

UI DESIGN PRIORITIES

Explainability
Every red finding links back to an exact graph path.

Honest status
Show confirmed, not reproduced, inconclusive separately.

Developer action
Provide repro steps rather than an unsupported risk score.

07 / EXECUTION PLAN

Build gates, roles & hackathon readiness

Prioritize one convincing end-to-end workflow; additional agents or dashboards are secondary.

MVP PRIORITIES

P0 / Must work	Synthetic 3-4 module app; one real buggy change; versioned baseline	Proof first
P0 / Must work	FalkorDB graph, linked invariant + test, graph-driven retrieval	Sponsor fit
P0 / Must work	Agent tool invocation, isolated test execution, evidence report	Core demo
P1 / After P0	Second scenario; abstention on missing coverage; agent memory	Robustness
P2 / Future	GitHub webhook, CI integration, scaled graphs, richer analytics	Portfolio growth

SUGGESTED TWO-PERSON OWNERSHIP

MEMBER A / ENGINE

FalkorDB schema + Cypher; diff parsing; scenario runner; verification logic; backend tests.

MEMBER B / EXPERIENCE

Frontend investigation view; API integration; fixtures + quality assurance; README and demo.

NON-NEGOTIABLE QUALITY GATES

1. A real code change triggers graph-selected scenario(s). **2.** Baseline passes and changed version reproducibly fails. **3.** Removing FalkorDB breaks automated graph-guided selection. **4.** Missing evidence returns inconclusive. **5.** A teammate can reproduce the result using documented steps.

HACKATHON SUBMISSION CHECKLIST

By 18 Oct 2026, 11:59 PM IST: Public repository; README and graph model; key Cypher queries / algorithms; working demo video; deployed application or full local setup; chosen track(s); disclosure of AI coding tools. Use public or synthetic data only. All claims and results must match working code.